

COMPUTER MUSIC REPRESENTATIONS AND MODELS

Homework No. 1 Report

Giuliano DI LORENZO
242712



POLITECNICO
MILANO 1863

1 Answers to question 1

1.1 GTZAN Dataset information

The GTZAN dataset is used to train algorithms of music genre classification and consists in a collection of 1000 mono audio tracks with 30 seconds duration. The audio files, which are wave files with 22050 Hz sample rate and 16-bit bit rate, are categorized into 10 genres, so that each one has 100 tracks.

The genres are mapped into numerical values from 0 to 9 as it follows:

- 0: pop
- 1: metal
- 2: classical
- 3: rock
- 4: blues
- 5: jazz
- 6: hiphop
- 7: reggae
- 8: disco
- 9: country

The code for mapping each genre numerical value to its name is given in the notebook, after the down-sampling factors definition.

The dataset is composed by two tensors: audio (rank-3 tensor) and genre (rank-2 tensor). The code renames them in *tensor_audio* and *tensor_genre* respectively and their shapes are (1000, None, 1) and (1000, 1). This means the audio tensor has 1000 tracks of slightly different duration, which explains the audio tensor dynamical shape. The genre tensor, on the other hand, has 1000 numerical elements (between 0 and 9) used to indicate the song genre.

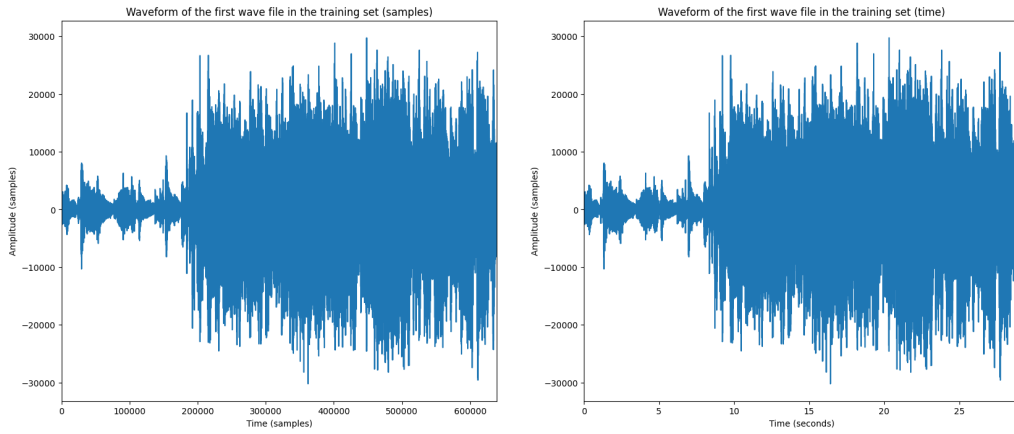
1.2 Tracks extraction

In order to develop a genre recognition algorithm based on rhythmic characteristics in a fast way and with low memory usage, the code extracts one every 10 songs to create a training set *audio_train* (from first to last song) and one every 52 songs to create a test set *audio_test* (from 12-th to last song).

Once defined the indices lists (*index_training* and *index_test*) for training and test respectively, the code creates two arrays (*genre_train* and *genre_test*) with the genre numerical values from the genre tensor for each extracted song, for training set and test set respectively. In this case, the genre extraction has occurred.

After that, the song extraction occurs, using the indices lists to select the songs from the audio tensor. Due the fact that every song has different samples number (audio tensor has dynamical shape along the second axis), the extracted samples go from 0 to *n_samples*. All songs are appended into the 2D arrays *audio_train* and *audio_test*, whose shapes are (100, 639450) and (20, 639450) respectively.

The plot of the first wave file of the *audio_train* set, which corresponds to the first pop song "Ironie" by Alanis Morissette, is given:

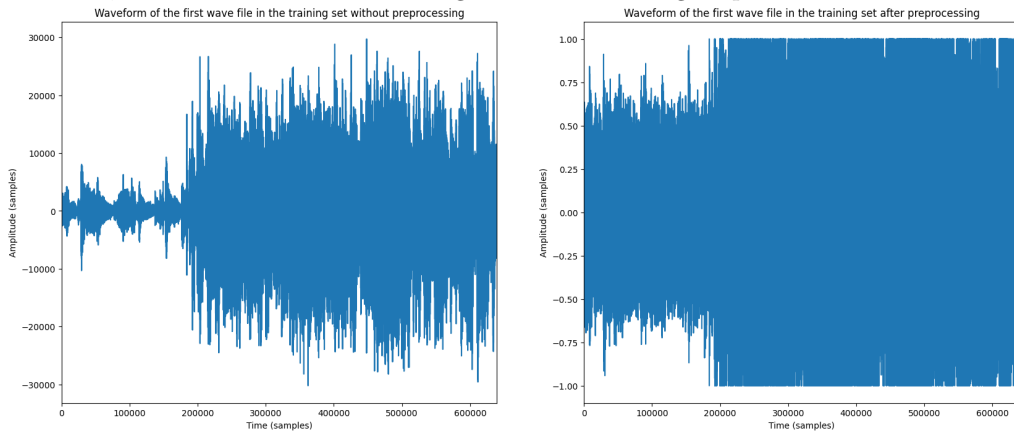


2 Answers to question 2

2.1 Pre-processing scaling

In order to make all files with the same characteristics, which allows the algorithm to work better, the code uses the *MinMaxScaler* library to limit the dynamic range of all songs between -1 and 1. In particular, this function maintains the linear relationship between data and ensures all data fall into the specific range of (-1, 1).

Consider that this elaboration doesn't keep the original sound dynamics, as a classic normalization would do, and that can be seen starting from the audio signal plot:



2.2 Phase novelty function

The rhythm recognition approach used for the genre classification starts from the phase novelty function. In order to implement this novelty function, the *compute_local_average* function is defined for the local average calculation, which will be subtracted to the novelty function, and the *principal_argument* function to map phase shifts into the $[-0.5, 0.5]$ range.

Once defined these two functions, the *compute_phase_novelty* calculates the novelty function of the audio signal. Inside this function, different other processes occur, like local average subtraction, half-wave rectification, normalization and plotting.

During the phase novelty computation, the code firstly determines the STFT of the audio signal, obtaining a matrix of frequency bins (rows) over time frames (columns), along with the novelty rate. From this, the phase information is obtained and normalized by 2π . Once obtained the phase, the code computes twice the discrete derivative over time by obtaining the differences between phase values of consecutive time instants (within same frequency). After every derivative, in order to avoid phase warping discontinuities, the code maps the phase differences using the *principal_argument* function. In the end, the code returns an array with all the phase information in time by summing all phase data in frequency (along rows). Due the fact that each derivative reduces the variable by one element, it is necessary to add two zeros at the end to obtain the same starting elements number.

2.3 Parameters influence

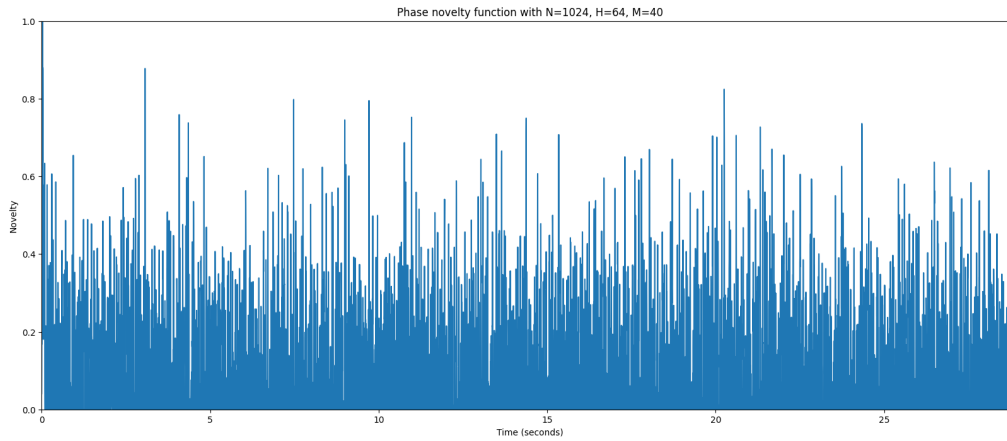
Three different parameters are used during the phase novelty computation (N, H, M) and it's possible to analyse their influence on the resulting plots.

N is the size (in samples) of the window for the STFT calculation and the block length in which the FFT is computed: higher values imply better frequency resolution at expenses of time resolution. So, the resulting novelty will have more samples for higher N values, which can affect the computational time.

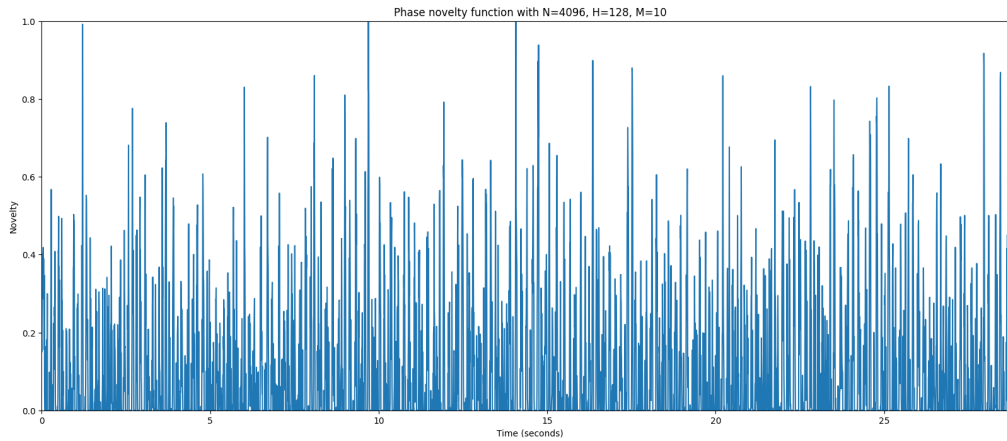
H is the hop size (in samples) for the window translation during the STFT: smaller values imply better time resolution at expenses of computational efficiency. Its choice may be related to the duration of the rhythmic events to analyse: smaller values allow to capture faster sound variations.

M is the size (in samples) of the window used to compute the local average: higher values imply smoother local averages at expenses of reactivity to rapid changes in the signal. Its choice may be related to the duration of the musical dynamic to consider: smaller values allow to capture faster sound variations.

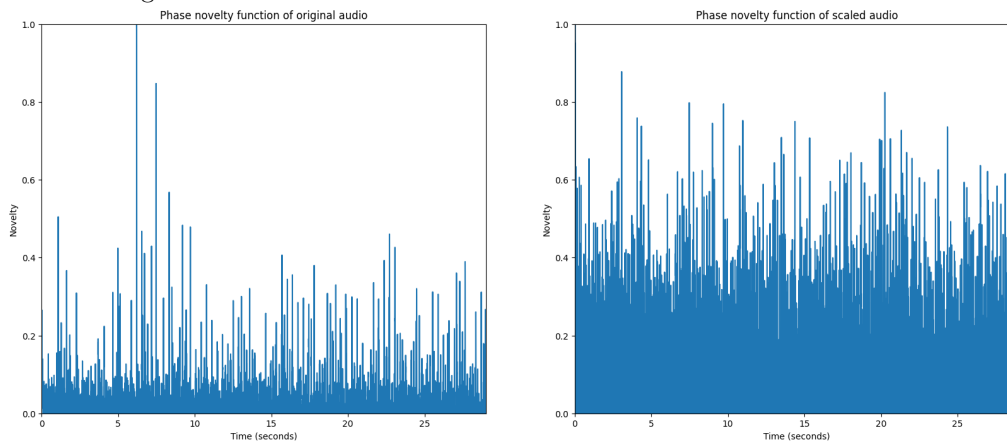
The plot of the phase novelty function on the scaled version of the first audio file is given:



Changing the parameters, it's possible to obtain slightly better results:



The phase novelty function plots can be considered quite noisy if related to the same plots obtained with the original audio files:



That can be given by the fact that the scaling process doesn't keep the original dynamics of the song. Beside that, it is possible, particularly when using different parameters' values, to notice certain onsets where the novelty assumes values near the maximum (which is 1, being already normalized).

The final goal is to obtain different rhythmic features for the audio tracks, and one of them is the phase novelty function, which describes phase variations in a sound track that can be related to rhythmic events.

3 Answers to question 3

3.1 Vector of rhythmic features

Once the tracks are well defined (extracted and normalized), the code starts computing all the rhythmic features of interest. In order to do that, a function *compute_feature_vector* is defined to create an array of rhythmic features for each song.

Firstly, we calculate the phase novelty function just defined. After that, we compute its standard deviation and mean, which describe how the values of the novelty tends to distribute and around which value.

Then, we calculate the tempogram, which is a matrix representation of the estimated BPM value of the tempo over the whole song's duration.

After that, it's of interest to evaluate the zero-crossing rate of the audio track, along with its standard deviation and mean values. The zero-crossing rate is the measure of how often a certain signal changes sign (and crosses the axis equal to zero), so it enhances where rapid changes occur in the audio track. In particular, the zero-crossing for the audio amplitude (amplitude equal to zero) can be associated with specific changes in the signal, that can be rhythmic in nature. So, intuitively, higher zero-crossing rates can describe faster changes within the audio track, which can be related to more frequent percussive events.

The next feature the code computes is the spectral flux, along with its standard deviation and mean values. The spectral flux is a measure of how much spectral changes occur over time, obtained by quantifying the amount of energy difference between frames. Like for the zero-crossing rate, high values of spectral flux can indicate more abrupt musical events, like rhythmic ones.

The last feature of interest is the tempo, which is the BPM value pace of the audio track.

Finally, every feature is appended into the array of features *f_vector*. Some of these audio characteristics (tempogram and zero-crossing rate) are matrixes, so they are reshaped into 1D arrays with the *ravel* function in order to get appended.

3.2 Feature vector's length

In the code, the spectral flux has been computed before tempogram and tempo so that these two features could be evaluated using the pre-computed spectral flux as parameter *onset_envelope*. Without this trick, the resulting feature vector is way longer, reaching more then 10M samples. This makes it impossible to use it for training the models later on, due to the excessive length.

Another way to solve the problem of feature vectors too long may be, for example, to downsample the tempogram, being the largest of the computed rhythmic characteristics. Although, doing so may not be ideal because removing content from a feature might results in less clear feature vectors. All of these rhythmic information are given into two 1D arrays *train_fvector* and *test_fvector* to keep the feature for the training set and the test set, respectively. Their shape will be (100, x) and (20, x), where x will be the length of all the features computed. In the code, x is equal to 2570448.

3.3 Features trade-off

The goal of the algorithm is to identify a song's genre starting from its rhythmic features. Due the fact that different audio tracks can have different rhythmic characteristics, it is best to implement an automatic search that uses a variety of audio information.

For example, a percussive pattern can be identified starting from the audio envelope features, while rhythmic information based on notes and chords can be noticed through a spectral analysis.

This is the reason why, in particular, both the zero-crossing rate and the spectral flux are computed. The first one is sensitive to envelope changes (time domain), so it can detect percussive events in the audio signal. On the other hand, the second one is sensitive to phase changes (frequency domain), so it can detect harmonic events in the audio signal.

4 Answers to question 4

4.1 SVMs and machine learning

Support Vectors Machines (SVMs) are algorithms for machine learning that can be used for classification and regression tasks. It is designed to find an hyperplane to separate in the best way the passed data points into different classes, by trying to maximise the distance between hyperplane and nearest data points. In this scenario, the data points to analyse are the rhythmic information inside the feature vectors.

The SVMs can identify particular relationships even in not-well separated data and can define the best hyperplane by using different kernels, which may work differently based on the data points given. Later in the code, will be possible to see how the different kernels work in the music genre classification using the rhythmic features defined before.

4.2 Models training

Firstly, the code trains a simple model using $C = 1$ and $kernel = "linear"$ and saves it with the name "svc_linear_C1_N_2048_H_128.joblib", in order to remember the values of C, kernel, N and H.

After training and saving the simplest model, the code implements a function for saving or loading new models with different parameters.

In order to define an SVM, it is necessary to call the function `sklearn.svm.SVC` with the wanted parameters (C and kernel). In order to train it, the code uses the function "fit" that requires the feature vectors (in the documentation written as X) and the labels of classification (in the documentation written as y), which are the genres in this case. Both the "fit" parameters must have same shape, that is why the code checks both the shapes of *train_fvector* and *genre_train* (they both have 100 along first axis).

It is possible to optimize the classification by using different hyperparameters, or in other words different settings' configurations for the machine learning algorithms. In the code, only the values of C and kernel will be analysed for the optimization process, but other settings can be changed (such as degree, gamma, coef0, ...).

In the code, every possible combination of C and kernel is evaluated, and its model is trained (if needed). The result will give 20 different trained models to try, in order to see how they respond both to training and test set. All of these models will be saved into an array *model_train*, in order to compute the accuracy values.

4.3 Training set accuracy

The code returns the accuracy value for every model previously trained using *accuracy_score*.

Analysing these results, it is possible to evaluate how C and kernel influence the training. In this case, the best kernel for the feature vectors is "linear".

The C value, which indicates how specific is the kernel's work, doesn't change the result for the linear kernel: this may indicate that an overfitting effect is occurring. The meaning is that the C value may not influence the training because the data points number or the feature vectors length are wide enough to train the model. In other words, the model is working with redundant information.

Intuitively, one way to avoid overfitting, and to avoid redundant computation, may be to slim the vector of features. Another one might be to reduce the number of training songs, though I reckon having a variety of songs may be generally a good thing, so the solution would be differentiating the training songs (keeping always the genre). Finally, more than slimming the feature vector may be useful to differentiate the kind of rhythmic features, so that the new optimal kernel could change from linear to another type. Each of these solutions have in common the result of changing the nature of the data points given to the SVM.

Finally in the code, just for better visualisation of the confusion matrixes, all the models with over 90% accuracy on the training set are saved into an array *best_models*.

5 Answers to question 5

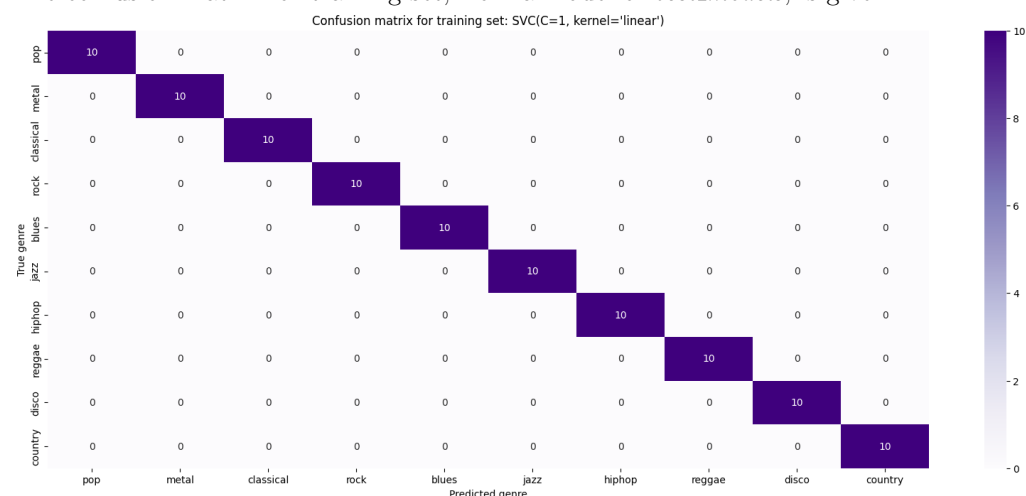
5.1 Test set accuracy and confusion matrix

Finally, the code tests each of the 20 trained models on the test set. Like in the training set, the model firstly makes a prediction starting by the given features. After that, the accuracy is computed by comparing predicted genres with known ones.

It is of interest to analyse the models with very high accuracy on the training set and the best accuracy on the test set, which are all and only the linear SVMs in this case. For these models (from *best_models*), the code plots the relative confusion matrix.

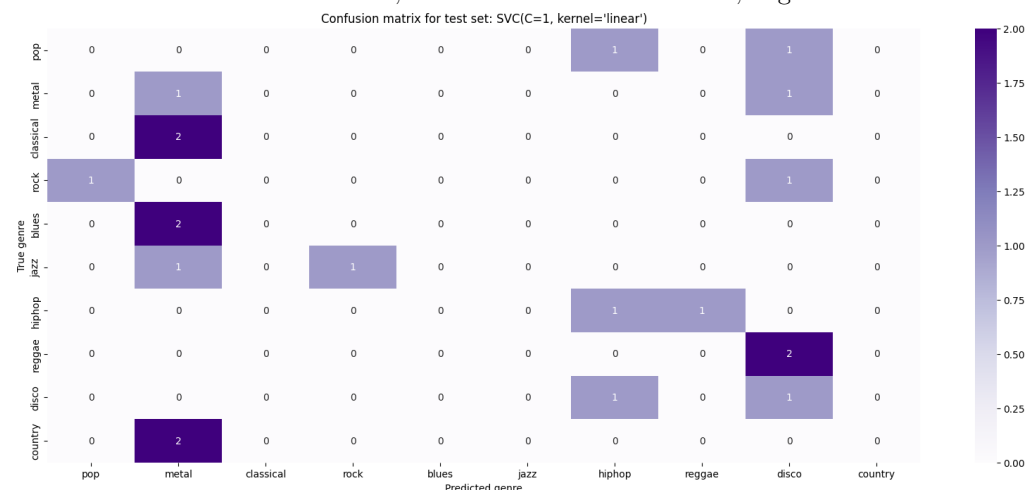
The confusion matrix of a classification model is a 2D representation of the performance of a SVM with the expected labels already known. In this scenario, the matrix visualizes the predicted genres (along x axis) and known genres (along y axis). The number of each block is the number of songs related to each combination of predicted-known genres. The confusion matrix is computed with the function *confusion_matrix* and is plotted with *heatmap*.

The confusion matrix for training set, from a model of *best_models*, is given:



All linear models have confusion matrix like this.

The confusion matrix for test set, from a model of *best_models*, is given:



All linear models have confusion matrix like this.

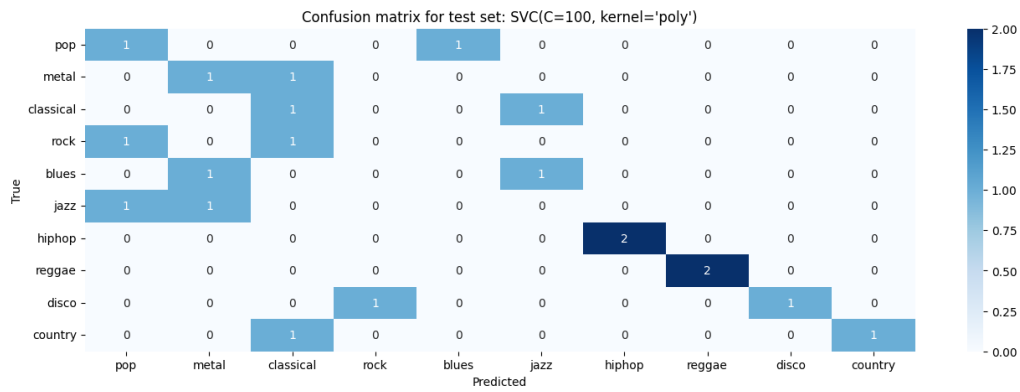
6 Conclusions

The kind of models that best classifies the musical genre of audio tracks is the linear, with 100% accuracy for the training set and 15% for the test set. This model recognised exactly 3 songs out of 20 (looking at the blocks along the diagonal): one for metal, one for hiphop, one for disco. In particular, this model may easily confuse a genre for metal or disco, likely.

From the previous questions, some conclusions may be drawn.

Regarding question 2, the scaling process may greatly influence the resulting accuracy. In fact, without scaling the audio tracks, the best accuracy obtained is 45% for the test set (and 93% for

the training set):



The logic behind this observation is that the scaling elaboration, which modifies the audio dynamics, results in rhythmic features less clear. Of course, it is strongly advised to limit the dynamic range in order to get more coherent audio features for the SVM to analyse.

Regarding question 3, the way of choosing the rhythmic features and their computation may influence the resulting accuracy. For example, considering one column every 17 and one row every 16 for the tempogram (before making the reshape), the best accuracy for the test set is 20%. Although, the reason of not doing that has already been explained.